

Laboratorio 2 - Arreglos y excepciones

FESTIVAL UNIVERSITARIO DE MÚSICA

Objetivo

- Aplicar arreglos básicos para almacenar una cantidad fija de objetos dentro de un programa.
- Acceder y modificar elementos de un arreglo mediante índices.
- Recorrer arreglos de objetos utilizando estructuras repetitivas y validar correctamente posiciones que contengan null.
- Utilizar ArrayList para almacenar y administrar una cantidad dinámica de objetos.
- Realizar operaciones de registro, búsqueda, modificación, eliminación y cálculo utilizando los datos almacenados.
- Implementar manejo de excepciones utilizando try-catch y finally.
- Utilizar excepciones para controlar entradas incorrectas y valores que no cumplan con las reglas establecidas.
- Aplicar de forma correcta los modificadores de visibilidad para no violar el encapsulamiento.
- Modelar el programa utilizando UML.
- Crear un programa principal que resuelva el problema utilizando programación orientada a objetos.

Introducción

La Universidad del Valle de Guatemala organizará un festival de música en el que diferentes artistas podrán presentarse ante la comunidad universitaria. Para organizar correctamente el evento, se necesita administrar los escenarios disponibles y mantener el registro de los artistas que participarán.

El festival cuenta con una cantidad limitada de espacios físicos destinados a escenarios. Algunos de estos espacios podrán encontrarse todavía sin configurar, por lo que las posiciones correspondientes podrán permanecer vacías. Debido a que la cantidad máxima de escenarios está definida desde el inicio, deberán administrarse utilizando un arreglo básico.

Por otro lado, la cantidad de artistas participantes puede cambiar durante la organización del festival. Nuevos artistas pueden incorporarse, modificar los datos de su presentación o cancelar su participación. Por esta razón, los artistas deberán almacenarse utilizando un ArrayList.

El sistema deberá permitir administrar ambas estructuras y manejar correctamente situaciones que puedan producir errores durante la ejecución del programa.

Para administrar el festival universitario se debe realizar lo siguiente:

1. Al iniciar el programa se debe solicitar la información del festival. Como mínimo se debe ingresar:
 - a. Nombre.
 - b. Código de identificación.
 - c. Nombre del coordinador.
2. Cada festival contará con un máximo de **5 escenarios**. Los escenarios configurados deberán almacenarse utilizando un **arreglo básico de objetos**.
3. Cada escenario deberá almacenar como mínimo:

- a. Código.
 - b. Nombre.
 - c. Ubicación.
 - d. Capacidad máxima de asistentes.
 - e. Estado.
4. La capacidad máxima de asistentes deberá ser mayor que 0. Si se intenta crear o modificar un escenario utilizando una capacidad menor o igual que 0, deberá generarse una `IllegalArgumentException`.
 5. Al configurar un nuevo escenario, el usuario deberá indicar la posición del arreglo donde desea almacenarlo. Antes de realizar la operación se deberá validar que:
 - a. La posición indicada se encuentre dentro de los límites del arreglo.
 - b. La posición seleccionada se encuentre disponible.
 - c. El escenario posea información válida.
 6. Las posiciones del arreglo que todavía no tengan un escenario configurado contendrán null. El programa deberá verificar esta condición antes de intentar utilizar los métodos de un escenario.
 7. El sistema deberá permitir consultar todos los escenarios configurados. No deberán mostrarse como escenarios existentes aquellas posiciones del arreglo que contengan null.
 8. El sistema deberá permitir consultar un escenario específico indicando la posición que ocupa dentro del arreglo. Si la posición no es válida o se encuentra vacía, deberá informarse al usuario sin finalizar inesperadamente el programa.
 9. El sistema deberá permitir modificar la capacidad máxima y el estado de un escenario previamente configurado. No se podrá modificar una posición que contenga null y la nueva capacidad deberá ser mayor que 0.
 10. El sistema deberá permitir retirar un escenario previamente configurado. Al retirarlo, la posición correspondiente deberá volver a contener null.
 11. Durante la organización del festival podrá registrarse una cantidad desconocida de artistas. Los artistas participantes deberán almacenarse utilizando obligatoriamente un `ArrayList`.
 12. Cada artista deberá almacenar como mínimo:
 - a. Código.
 - b. Nombre artístico.
 - c. Género musical.
 - d. Duración de la presentación en minutos.
 - e. Cantidad estimada de asistentes.
 13. La duración de la presentación deberá ser mayor que 0 y la cantidad estimada de asistentes no podrá ser negativa. Si alguno de estos valores no cumple con las reglas establecidas, deberá generarse una `IllegalArgumentException`.
 14. El sistema deberá permitir registrar nuevos artistas utilizando el `ArrayList`. No se podrán registrar dos artistas con el mismo código.
 15. El sistema deberá permitir consultar todos los artistas registrados. Si todavía no existen artistas, deberá manejarse correctamente la situación.
 16. El sistema deberá permitir buscar un artista mediante su código. Para realizar la búsqueda deberá recorrerse el `ArrayList`.
 17. El sistema deberá permitir modificar la información de un artista previamente registrado. Los nuevos valores deberán cumplir las mismas validaciones utilizadas durante su creación.
 18. El sistema deberá permitir cancelar la participación de un artista. Para ello deberá

- localizarse al artista correspondiente y eliminarlo del ArrayList.
19. El sistema deberá calcular y mostrar:
 - a. Cantidad de escenarios configurados.
 - b. Cantidad de espacios disponibles para escenarios.
 - c. Escenario con mayor capacidad máxima de asistentes.
 - d. Cantidad de artistas registrados.
 - e. Artista con la presentación de mayor duración.
 - f. Artista con mayor cantidad estimada de asistentes.
 - g. Promedio de duración de las presentaciones registradas.
 20. Para los cálculos relacionados con artistas únicamente deberán considerarse los objetos registrados dentro del ArrayList. Si este se encuentra vacío, el programa deberá evitar realizar cálculos que requieran elementos existentes.
 21. Cuando el programa solicite un valor numérico mediante Scanner y el usuario ingrese un dato que no corresponda al tipo solicitado, deberá manejarse la `InputMismatchException`. El programa deberá limpiar la entrada incorrecta y permitir que el usuario continúe utilizando el sistema.
 22. El programa deberá utilizar bloques try-catch para manejar las excepciones que puedan producirse durante la interacción con el usuario y las validaciones de los objetos.
 23. El programa deberá utilizar al menos un bloque finally dentro de una operación donde tenga sentido ejecutar una acción final independientemente de que la operación se complete correctamente o se produzca una excepción.
 24. El programa deberá continuar funcionando después de una entrada incorrecta o una excepción que haya sido manejada.

Ejercicio

Análisis

Responda las siguientes preguntas:

1. ¿Qué propiedades y métodos tendrá cada clase?
2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?
3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico? ¿Qué tipo de objetos almacenará y cuál será su tamaño?
4. ¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?
5. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?
6. ¿Qué parámetros serán requeridos por los métodos en sus clases?
7. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?
8. ¿Cómo determinará si una posición del arreglo contiene un escenario o contiene null?
9. ¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?
10. ¿Qué situaciones del programa pueden producir excepciones? Identifique qué excepciones deberán manejarse y en qué partes del programa utilizará try-catch y finally.

Diseño

Desarrolle un diagrama de clases que muestre los miembros de cada clase y que establezca las relaciones necesarias entre ellas.

Incluya en su diagrama al driver program, que contiene el método main().

El diagrama deberá representar correctamente los atributos, métodos, tipos de datos, parámetros y modificadores de visibilidad identificados durante el análisis.

El diagrama deberá representar correctamente las propiedades que utilizan el arreglo básico y el ArrayList.

Use anotaciones (recuadros de texto) en su diagrama para aclarar cualquier información que considere necesaria.

Programa

Desarrolle un sistema que contenga un driver program y las instancias necesarias de las clases identificadas durante su análisis.

El programa deberá utilizar como mínimo un **arreglo básico de objetos** para almacenar los escenarios configurados en el festival y un **ArrayList de objetos** para almacenar los artistas registrados.

El programa deberá implementar manejo de excepciones mediante try-catch y finally, de acuerdo con las situaciones identificadas durante el análisis.

Así mismo, debe mostrar un menú con las siguientes opciones:

1. **Nuevo festival:** reemplaza la instancia actual del festival. Al crear un nuevo festival, este comienza sin escenarios configurados y sin artistas registrados.
2. **Configurar escenario:** solicita la posición donde se desea configurar el escenario y la información necesaria para crearlo. La posición debe estar disponible.
3. **Consultar escenarios:** muestra todos los escenarios configurados actualmente y las posiciones que ocupan dentro del arreglo.
4. **Consultar un escenario:** solicita una posición y muestra la información del escenario almacenado en ella. Si la posición está vacía, deberá indicarse al usuario.
5. **Modificar escenario:** solicita la posición de un escenario previamente configurado y permite modificar su capacidad máxima y estado.
6. **Retirar escenario:** solicita la posición de un escenario y lo elimina del arreglo, dejando nuevamente disponible dicha posición.
7. **Registrar artista:** solicita la información de un nuevo artista y lo agrega al ArrayList. No deberá permitirse registrar códigos repetidos.
8. **Consultar artistas:** muestra todos los artistas registrados para el festival.
9. **Buscar artista:** solicita el código de un artista y muestra su información si se encuentra registrado.
10. **Modificar artista:** solicita el código de un artista previamente registrado y permite modificar su información.

11. **Cancelar participación:** solicita el código de un artista y lo elimina del ArrayList.
12. **Mostrar reporte del festival:** muestra la cantidad de escenarios configurados y espacios disponibles, el escenario con mayor capacidad, la cantidad de artistas registrados, el artista con la presentación de mayor duración, el artista con mayor cantidad estimada de asistentes y el promedio de duración de las presentaciones.
13. **Salir:** termina la ejecución del programa.

El programa deberá validar las entradas necesarias para evitar almacenar información inválida, acceder a posiciones inexistentes del arreglo, utilizar referencias null o realizar operaciones inválidas sobre un ArrayList vacío.

Las entradas numéricas incorrectas deberán manejarse mediante `InputMismatchException`. Los valores que no cumplan con las reglas establecidas para los objetos deberán manejarse mediante `IllegalArgumentException`.

El programa deberá utilizar `ArrayList` directamente para la colección dinámica. **No deberá utilizarse `List`.**

Material a entregar en Canvas

- Archivo PDF con el análisis y diseño.
 - No incluya únicamente la URL del diagrama; adjunte el diagrama dentro del PDF o como un archivo externo.
 - Use la plantilla proporcionada.
- URL al repositorio público en GitHub conteniendo todos los archivos .java necesarios para ejecutar el programa.

Evaluación

- **Análisis (30 puntos)**
 - **(5 puntos)** Se listan correctamente todos los requisitos funcionales del sistema a desarrollar, punto por punto.
 - **(5 puntos)** Se describe el propósito de cada una de las clases identificadas. La descripción es lógica y coherente con lo que hace la clase dentro del sistema que modela y resuelve el problema presentado.
 - **(10 puntos)** Se describe el propósito de cada atributo de cada clase. La descripción es suficiente para comprender la razón de la creación del atributo y su importancia como propiedad de las instancias o clases. Se identifican correctamente las propiedades que utilizan el arreglo básico y el ArrayList.
 - **(10 puntos)** Se describe el propósito de la creación de cada método de cada clase. La descripción es suficiente para comprender qué acciones realizará cada método, qué información requerirá para funcionar y qué resultados producirá. Se identifican las operaciones que pueden producir excepciones.
- **Diseño (30 puntos)**
 - **(8 puntos)** La representación de las clases coincide con el estándar del lenguaje UML.
 - **(12 puntos)** Están representados cada uno de los componentes de las clases de manera correcta y coinciden con lo descrito en el análisis.
 - **(10 puntos)** Las relaciones entre las clases están correctamente representadas.
- **Programa (40 puntos)**
 - **(10 puntos)** La cantidad de clases identificadas es suficiente para resolver el problema planteado y se utilizan correctamente los principios básicos de

programación orientada a objetos.

- **(10 puntos)** Se implementa correctamente un arreglo básico de objetos para almacenar los escenarios y se realizan las operaciones solicitadas utilizando índices, estructuras repetitivas y validación de posiciones null.
- **(10 puntos)** Se implementa correctamente un ArrayList para almacenar los artistas y se realizan las operaciones solicitadas de registro, búsqueda, modificación, eliminación y cálculo.
- **(10 puntos)** Se implementa correctamente el manejo de excepciones utilizando try-catch y finally. Se manejan las entradas incorrectas y los valores inválidos sin provocar la finalización inesperada del programa.